

Scheda — FI2 LAB03: ADT Frazione, seconda parte

Materiale: LAB-03-Frazione-SecondaParte.pdf (sl. 1-10), Lab03-Frazione-SecondaParte-Startkit.zip · **Modalità:** guidata **Teoria che entra qui:** 04b (in corso) — Frazione *all together* sl. 85; 02 — compatibilità fra reali sl. 54-55 (per `getDouble`).

Setup

- lo startkit contiene già Frazione **nella versione del docente** di LAB02, MyMath con mcm completo, FrazioneTest, MainFrazione [sl. 10]: si parte da quello, non dal tuo LAB02 (è la forma «da seguire» del confronto);
- import *Existing Projects* → **Finish**, poi **Refactor** → **Rename** con nome diverso dalla cartella; default package; -ea nelle VM arguments di FrazioneTest;
- le X rosse iniziali sono i metodi nuovi che i test chiamano e Frazione non ha ancora.

Ordine di lavoro

#	Metodo (firme da UML sl. 7)	Cosa restituisce	Test	Costrutto nuovo → slide
1	<code>sum(Frazione f): Frazione</code>	nuova frazione, ridotta	«somma fra due frazioni»	è scritto sulla slide (sl. 6): capiscilo, poi scrivilo senza guardare
2	<code>sumWithMcm(Frazione f): Frazione</code>	stessa somma, via <code>MyMath.mcm</code>	«somma con mcm», «uguaglianza...»	f.den: campo private di un altro oggetto — sl. 6
3	<code>sub(Frazione f): Frazione</code>	differenza ridotta	«sub»	—
4	<code>mul(Frazione f): Frazione</code>	prodotto ridotto	«prodotto»	—
5	<code>reciprocal(): Frazione</code>	reciproco, ridotto	«reciproco»	—
6	<code>div(Frazione f): Frazione</code>	quoziente ridotto	«divisione»	riuso di metodi già scritti
7	<code>compareTo(Frazione f): int</code>	0, 1 / -1 [sl. 5]	un solo assert	—
8	<code>getDouble(): double</code>	valore reale	<code>== 0.25</code>	conversione <code>int</code> → <code>double</code> — 02 sl. 54-55

Casi limite da pensare

- Sl. 6 accede a `f.den` anche se `den` è `private`: `private` protegge **dalla classe o dall'oggetto**?
- `sub` e `div`: quanto codice nuovo serve davvero, se hai già `sum`, `mul`, `reciprocal` e il costruttore?
- `reciprocal` di $-2/3$: il segno finisce sul denominatore? Chi lo rimette a posto?
- `compareTo` confronta moltiplicando in croce: perché il verso della disuguaglianza è giusto **solo** perché il denominatore è sempre positivo? (Δ pattern 5, verso.)
- `compareTo`: il test controlla solo il caso 0. Chi controlla 1 e -1 ? (Δ pattern 2: test verde $\neq$ metodo corretto — prova tu $1/2$ contro $1/3$ e viceversa.)
- `getDouble` di $3/12$: se il risultato è 0.0 , che divisione hai fatto?
- `reciprocal` di $0/5$, `div` per $0/1$: cosa succede? Sl. 9 dice che «per ora» non si sa gestirlo — annota cosa osservi, non risolverlo.
- `toString` deve dare `"3/12"`: non ridurre dentro `toString`.

Test rosso?

Non compila (firma, nome, tipo di ritorno) · `AssertionError` = failure: la riga indica l'assert · altra eccezione (`/ by zero`) = error → prontuario §1, oppure scrivilo in chat. Silenzio = verde **solo con -ea**.

Dopo i test verdi

- copia il progetto in `corsi/FI2/esame_FI2/svolti/LAB03_Frazione/`;
- confronto con la soluzione del docente → `confronto_LAB03.md`, cercando le **ri-dondanze** (criterio «questo lo so già?», `confronto_LAB02.md` punto 5);
- `/chiudi` aggiorna il prontuario.